

OPEN-HARDWARE CORE-ET · STATUS UPDATE

ET-SoC-1

Architecture overview, progress so far, and the road ahead

[Presenter] · September 2026

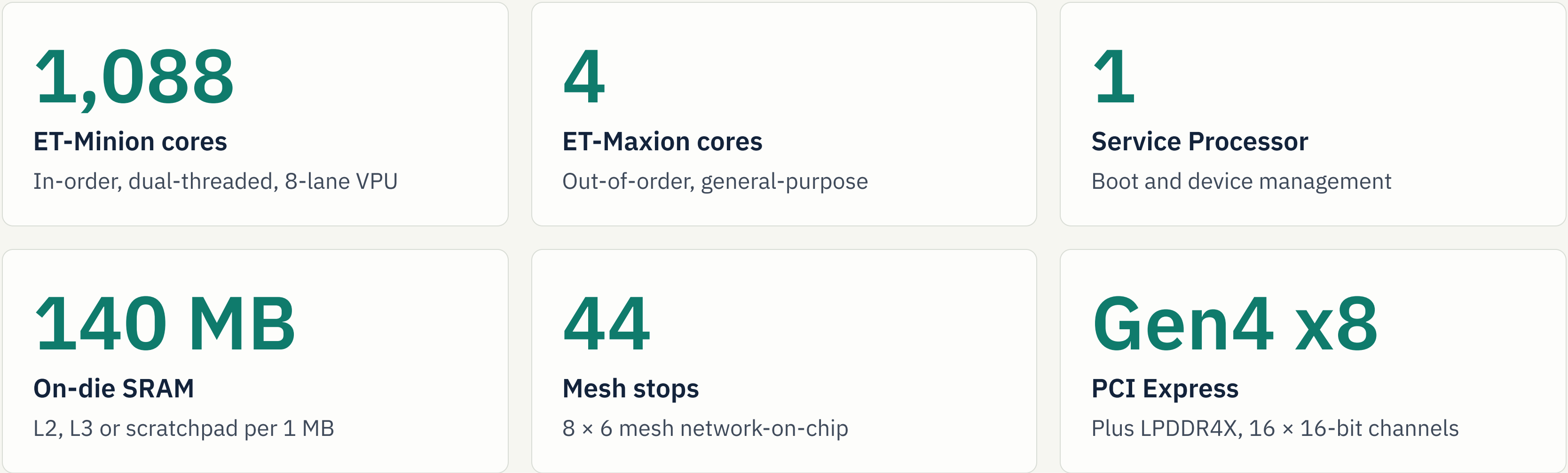
01

ET-SoC-1 overview

A many-core RISC-V SoC for AI/ML inference, and the part of it that is open source

ET-SoC-1 at a glance

1,093 general-purpose 64-bit RISC-V cores on a mesh network-on-chip, built for AI/ML inference



The ET-Minion hierarchy

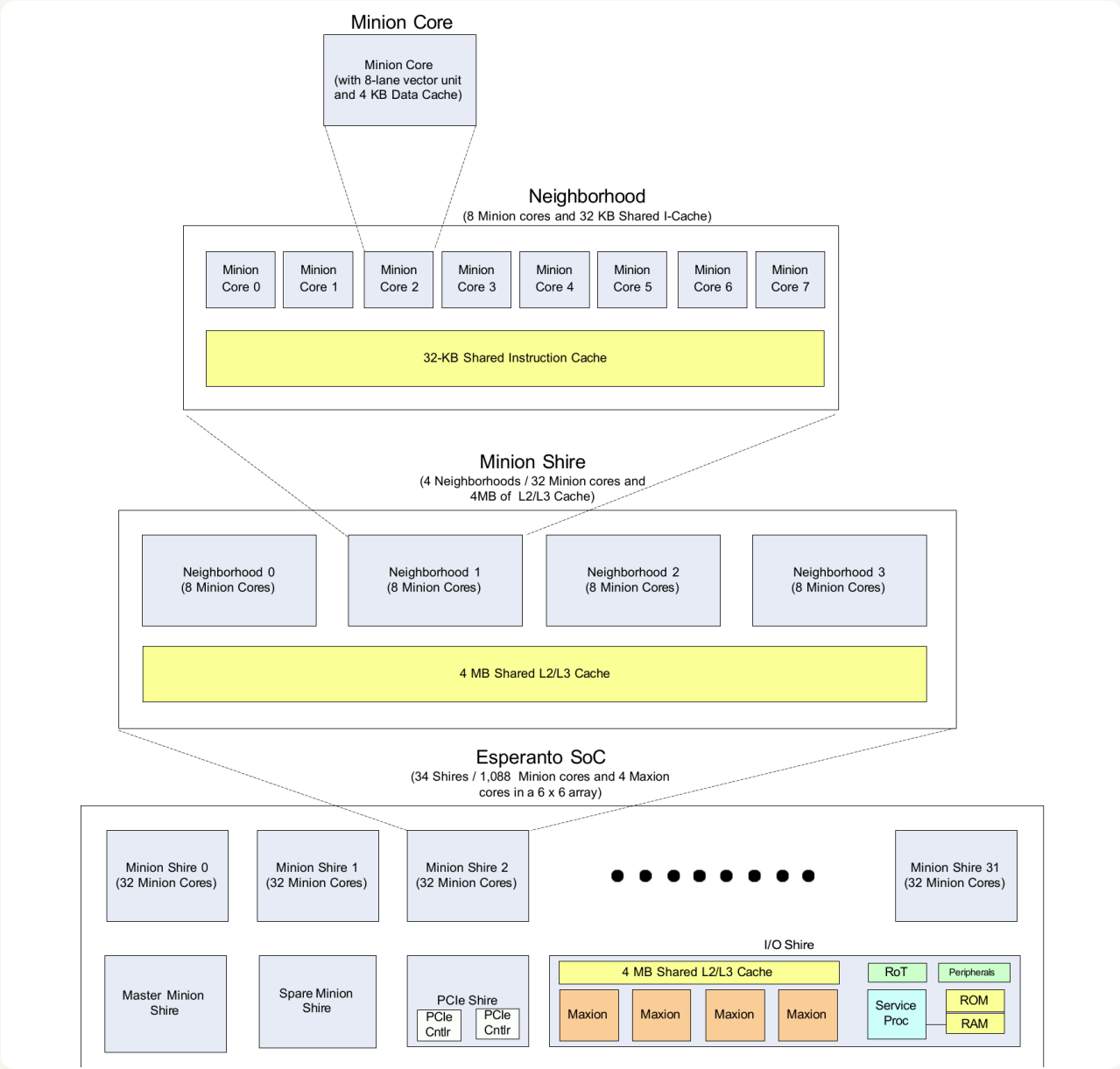
- ET-Minion core

RV64IMFC, 2 harts, 4 KB L1 data cache, 8-lane VPU
- Neighborhood · 8 cores

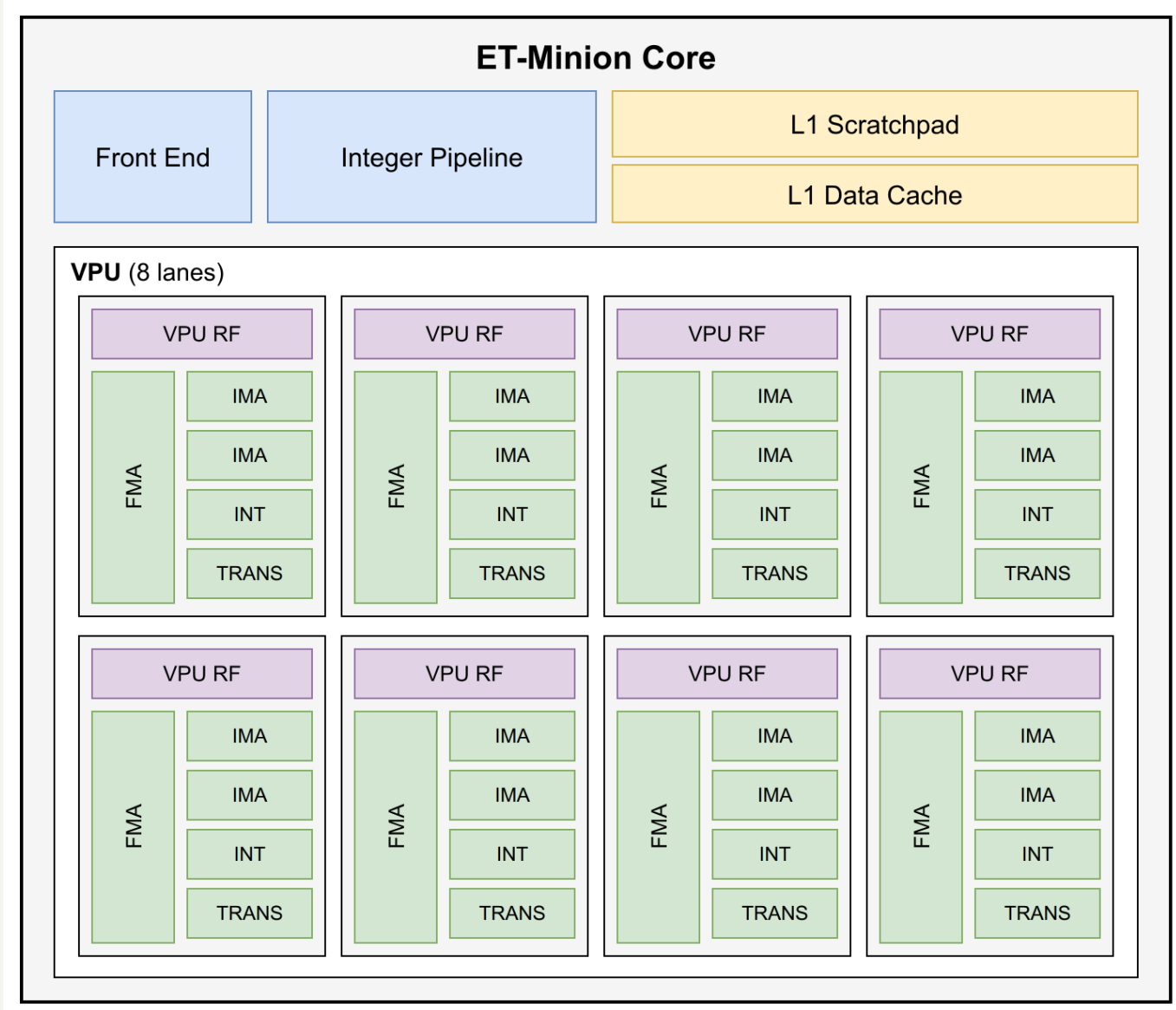
Shared 32 KB L1 I-cache, 2 L0 micro-caches, 2 PTWs
- Minion Shire · 32 cores

4 Neighborhoods, 4 MB Shire Cache, crossbar, mesh stop
- Shire array · 34 Shires

32 compute, 1 master (management), 1 spare



The ET-Minion core



FRONT END

Fetch, thread scheduling, RVC expansion and decode

INTEGER PIPELINE

RV64IMFC execution for two harts, CSRs and traps

L1 DATA CACHE

4 KB per core, cache or scratchpad mode

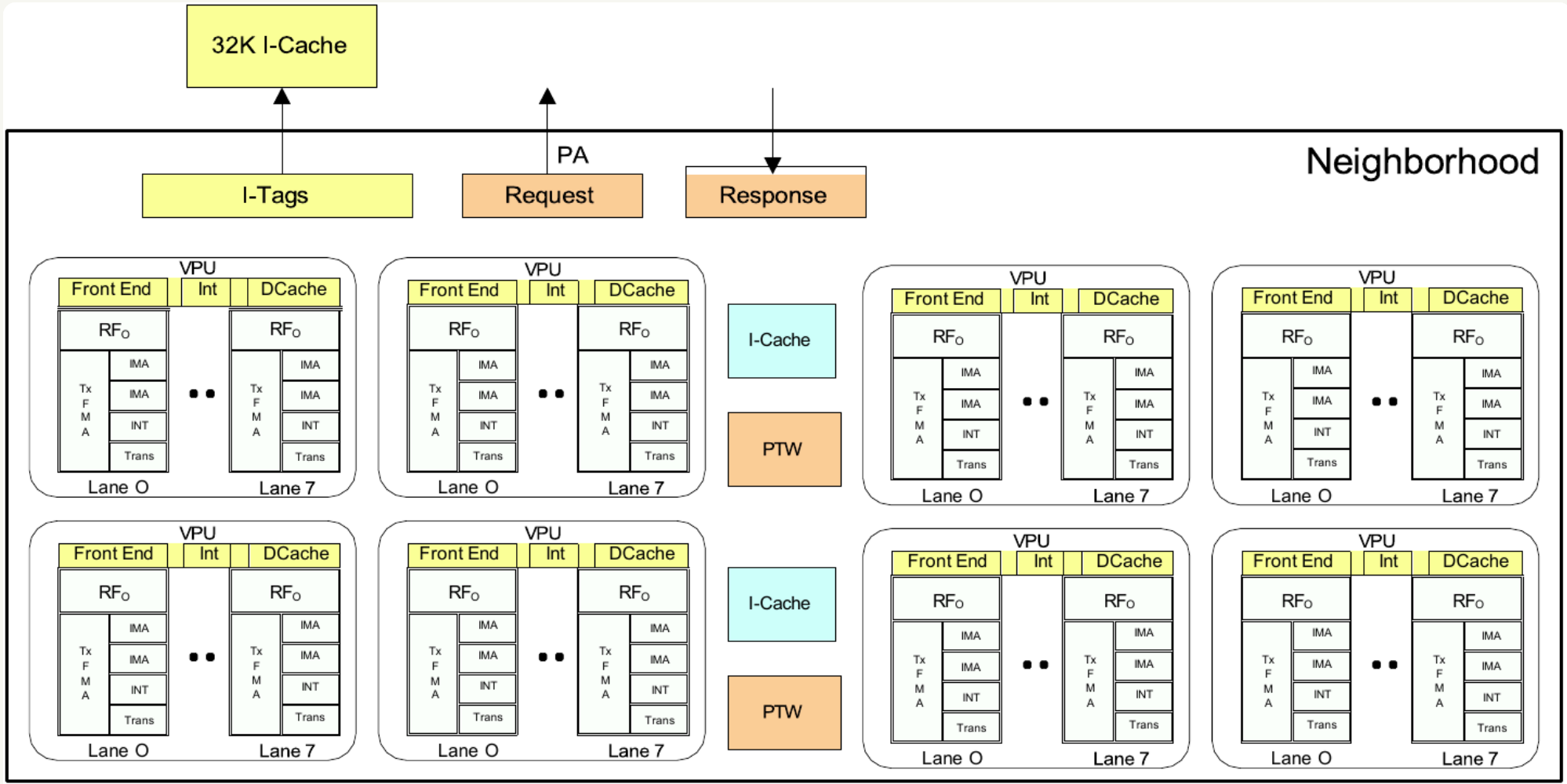
VPU

8 lock-step lanes: FMA, 2 × IMA, INT, TRANS

EXTENSIONS

Vector, tensor, atomic, message, cache-mgmt, fast sync

The Neighborhood: eight cores, shared resources



32 KB L1 I-cache

128 sets × 4 ways, shared by all 8 cores

2 × L0 micro-cache

16 fully associative lines, each shared by 4 cores

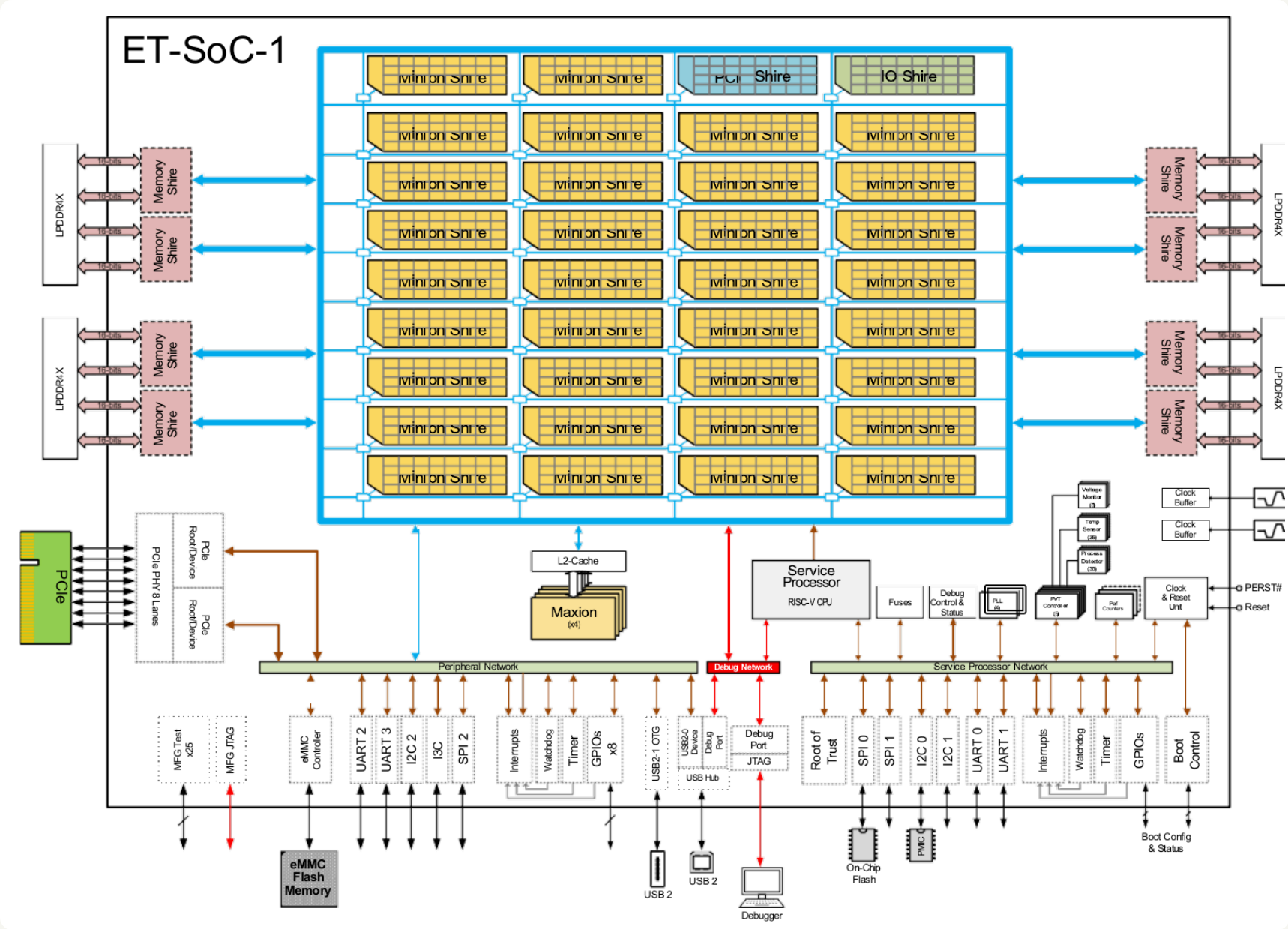
2 × page-table walker

Each serves 4 data caches and the I-cache side

PMU

12 × 64-bit counters: 8 core, 4 Neighborhood events

Chip organization on the mesh



Shire	Count	Contents
Minion	34	32 ET-Minions, 4 MB cache
Memory	8	LPDDR4X controller
PCIe	1	Two Gen4 controllers
I/O	1	ET-Maxions, SP, peripherals
Total	44	Mesh stops

8 × 6 grid; the corners are empty because each side holds only four Memory Shires.

What is open source: the CORE-ET CPU Subsystem

One Neighborhood, plus the logic that replaces the Minion Shire and mesh, integrated through an AXI master port and an APB slave port

Neighborhood

8 ET-Minions, 16 harts, shared I-cache, 2 PTWs

ETL2AXI bridge

ET-Link requests to AXI transactions

PLIC

External interrupts to the 16 harts

CLINT

Machine timer and timer interrupts

FLB / FCC

Fast local barrier and credit counters

ESRs + APB mux

Configuration through the APB slave port

Reset

Cold, warm and debug-module resets

Power-down

Request/acknowledge, subsystem and per core

02

Progress so far

The blocks studied in the RTL, and the documentation groundwork

Blocks understood in the RTL

minion/frontend

Front End

- Thread scheduling across the two harts
- Per-thread instruction buffers
- RVC expansion and decode to the integer pipeline and VPU
- Request and response interface to the L0 micro-cache

neigh/icache

Instruction caches

- L0 micro-cache: 16 lines, one per 4 cores
- L1 I-cache: 32 KB, 128 sets × 4 ways
- Arbitration between the two L0s on a miss
- Data RAM outside the Neighborhood

minion/vpu

VPU control path

- Front-end VPU decoder for directly issued instructions
- Tensor unit controller (ML unit): TFMA, TQuant, TStore/TReduce
- Transcendental FSM for exp and log sequences
- u-instruction decoder driving the 8 lanes

Documentation groundwork

01 Legacy docs made searchable

13 u-arch PDFs converted to Markdown, word for word, with their figures

02 Document plan agreed

13 chapters that follow the RTL hierarchy, core to CPU Subsystem

03 Chapter 1 drafted

ET-SoC-1 Overview, with every statement traced to the RTL or Esperanto docs

04 One source, three outputs

AsciiDoc source that builds HTML, PDF and DOCX

03

Future goals

A detailed u-arch document, RISC-V Vector 1.0 in the VPU, and low-precision tensor math

GOAL 1

A detailed micro-architecture document

Principles

- RTL is the source of truth; differences from the legacy docs are recorded
- Every statement traced to the RTL or Esperanto documents
- Written for RTL designers and DV engineers
- Per block: I/O and struct tables, pipelines, FSMs and timing diagrams

Status: Chapter 1 drafted

CH 2-6 · ET-MINION CORE

Overview, Instruction Fetch, Integer Pipeline, Data Cache, VPU

CH 7-12 · NEIGHBORHOOD

Overview, L0 and L1 I-cache, PTW, request/response path, tensor support, sync, registers and PMU

CH 13 · CPU SUBSYSTEM

ETL2AXI bridge, PLIC, CLINT, FLB/FCC, registers and APB, reset and power-down

RISC-V Vector 1.0 compliance for the VPU

Extend the existing VPU to run RVV 1.0 while keeping every Esperanto custom instruction.

The eight 32-bit lanes already form one 256-bit register per thread, so the register file maps directly onto VLEN = 256.

Initial scope

Item	Target
Instructions	Vector floating-point
SEW	16 and 32 bits
LMUL	1 first, then > 1 via a sequencer
VLEN	256 bits (existing register file)
Deferred	Vector integer: lanes lack INT16/INT32

Four hardware enhancements

01

Dedicated FP register file

64 × 32-bit, 32 per thread, so scalar FP no longer borrows the low 32 bits of each 256-bit register

02

Dedicated scoreboard

Dependency tracking for the new FP register file, used by both the VPU and the integer pipeline

03

Vector CSRs and state

vstart, vxsat, vxrm, vcsr, vl, vtype, vlenb; vsetvl(i); context switch and exception resume

04

Vector micro-instruction sequencer

Issues one operation per register of an LMUL group and masks lanes from vl

GOAL 3 • FUTURE OPTIMIZATION

FP8 and FP4 tensor math in the TXFMA units

Smaller formats give more operations per cycle and less data movement for inference. NVIDIA Blackwell (2024) and AMD Instinct MI350 (2025) already support FP8 and FP4.

Next steps, in order

STEP 1 · IN PROGRESS

Micro-architecture document

- ET-Minion core, Chapters 2–6
- Neighborhood, Chapters 7–12
- CPU Subsystem, Chapter 13



STEP 2

RVV 1.0 VPU upgrade

- FP register file and scoreboard
- Vector CSRs and state
- Micro-instruction sequencer



STEP 3

Low-precision tensor math

- FP8 and FP4 in the TXFMA units
- Tensor FMA and MAC

Questions and discussion

Feedback on the document plan and the VPU upgrade scope is welcome.

[\[Presenter\]](#) · [\[Contact\]](#)